Review notes · for you

Functions, lists and algorithms: unit check

Programming · Level 3 · Unit check

Time	Materials
20 min	Paper and a pen. No device needed.

Learning goals

- Functions
- Lists and loops
- Searching
- Sorting

Activity: Break my function

1. Warm-up (3 min): Write one idea from each of the four lessons in your own words.
2. Write (7 min): Write a small function from this unit (largest, find, count_above or is_sorted) with three asserts.
3. Break (7 min): Swap with a partner and write one edge-case assert that you think will break their function. Run it.
4. Fix (3 min): If yours broke, fix it and run every assert again.

Discussion

- Which algorithm surprised you most?
- Which edge case broke the most code today?
- When would you use a set instead of a sorted list?

Check yourself

- One algorithm I can now write and test from memory

Try it this week

Choose one algorithm from this unit (find, binary search, is_sorted or a sort) and write it from memory, with three asserts. Then compare it with the lesson.

Quiz sheet

Functions, lists and algorithms: unit check

Programming · Level 3 · Unit check

Name _____ Class _____ Date _____

1 **What does this program print?**

```
def f(x):
    return x * 2

print(f(3) + 1)
```

◯ 7 ◯ 6 ◯ An error ◯ None

2 **A function that prints its answer, with no return, gives back that answer.**

◯ True ◯ False

3 **Write count_above(values, limit): it returns how many values are greater than limit.**

```
def count_above(values, limit):
    # how many values > limit
    return 0
```

Write your program:

4 **total should return the sum of the list, but for [4, 6, 8] it returns 4. Which line is wrong?**

Circle the line with the mistake. Write it correctly:

```
1  def total(values):
2      t = 0
3      for v in values:
4          t = t + v
5          return t
```

5 **Match each algorithm to its worst-case work for 1,000 items.**

Linear search	• • • •	about 500,000 comparisons
Binary search	• • • •	1,000 looks
Bubble sort	• • • •	about 10,000 comparisons
Merge sort	• • • •	about 10 looks

6 **A sorted list has a million names. At most, about how many looks does binary search need?**

◯ About 20 ◯ About 1,000 ◯ About 500,000

7 **Put the lines in order: push A, then B, onto a list used as a stack, then pop the top item and print it (B). One line doesn't belong.**

Number the lines in order (1, 2, 3…). Cross out any line that doesn't belong.

☐ `print(top)`

☐ `top = stack.pop()`

☐ `stack = []`

☐ `top = stack.pop(0)`

☐ `stack.append("B")`

☐ `stack.append("A")`

8 **Which are edge cases worth testing for a sorting function?**

☐ A list with one item ☐ Another list of five random numbers ☐ A list with repeated values

☐ An empty list

9 **Fill the gaps: last is the last reading, and first_two the first two.**

```
temps = [21, 25, 31, 28]
last = temps[ _____ ]
first_two = temps[ _____ :2]
```

Word bank: `0` `1` `-1` `4`

10 **Insertion sort is fast on a list that is already nearly sorted.**

◯ True ◯ False

11 **Put the steps of testing a function in order.**

☐ Run them

☐ Fix the function until every assert passes

☐ Decide what it should return for each input

☐ Write asserts, including edge cases

Check your answers

Functions, lists and algorithms: unit check

Programming · Level 3 · Unit check

1 **7**

f(3) returns 6, so f(3) + 1 is 7. Because f returns its result, it can be used in a sum.

2 **False**

Without return, Python hands back None: print only shows the answer on the screen.

3

```
def count_above(values, limit):
    count = 0
    for v in values:
        if v > limit:
            count = count + 1
    return count
```

A counter in a loop, returned after the loop.

4 **Line 5: `return t`**

The return is indented inside the loop, so the function returns after the first item.

5 **Linear search — 1,000 looks**
Binary search — about 10 looks
Bubble sort — about 500,000 comparisons
Merge sort — about 10,000 comparisons

n, log n, n × n ÷ 2 and n × log n: the same 1,000 items, very different amounts of work.

6 **About 20**

Halving a million reaches 1 in about 20 steps.

7 **Top to bottom: 5, 4, 1, X, 3, 2**

```
stack = []
stack.append("A")
stack.append("B")
top = stack.pop()
print(top)
```

A stack takes from the end: pop() gives the last item added, B.

8 **An empty list; A list with one item; A list with repeated values**

Empty, single and repeated values are where sorts break.

9

```
temps = [21, 25, 31, 28]
last = temps[-1]
first_two = temps[0:2]
```

-1 counts from the end; a slice from 0 up to, but not including, 2 gives the first two.

10 **True**

It only moves items that are out of place, so a nearly sorted list takes about n steps.

11 **Top to bottom: 3, 4, 1, 2**

Deciding the right answers first, edge cases included, is what makes the tests worth running.